9강. 성능 최적화 & 전역 관리

React

Hook – useRef()

- **useRef() 훅**

  컴포넌트에서 **값을 저장하거나 DOM 요소에 직접 접근할 때 사용하는 훅(Hook)**입니다.

```
const ref = useRef(initialValue);
```

- ref.current에 값이 저장됨 - **{current: value}**
- 값이 변경되어도 컴포넌트가 다시 렌더링되지 않음

사용 예)

- 입력창 자동 포커스
- 이전 값 저장
- 타이머 ID 저장 (setTimeout, setInterval)
- 외부 라이브러리 DOM 제어

Hook – useRef()

- 값 저장(렌더링과 무관한 데이터)

```
렌더링...
▸ {current: 0}
렌더링...
▸ {current: 0}
렌더링...
▸ {current: 0}
Ref: 1
Ref: 2
Ref: 3
Var: 1
Var: 2
Var: 3
```

Hook – useRef()

- 값 저장(렌더링과 무관한 데이터) – Counter.jsx

```
function Counter() {
  const [count, setCount] = useState(0);
  // useRef 훅을 사용하여 countRef라는
  // 참조 객체를 생성하고 초기값을 0으로 설정
  const countRef = useRef(0);
  // 일반 변수 - 컴포넌트가 리렌더링될 때마다 초기화
  let countVar = 0;

  console.log("렌더링...");
  // {current: 0} 객체 - 초기값은 0입니다.
  // countRef.current를 사용
  console.log(countRef);

  // 상태 증가
  const increaseCount = () => {
    setCount(count + 1);
  }
```

Hook – useRef()

- 값 저장(렌더링과 무관한 데이터)

```
// 참조 증가
const increaseCountRef = () => {
  countRef.current = countRef.current + 1;
  console.log("Ref:", countRef.current);
}

// 일반 변수 증가
const increaseCountVar = () => {
  countVar = countVar + 1;
  console.log("Var:", countVar);
}

return (
  <div>
    <p>State: {count}</p>
    <p>Ref: {countRef.current}</p>
    <p>Var: {countVar}</p>

    <button onClick={increaseCount}>State 증가</button>
    <button onClick={increaseCountRef}>Ref 증가</button>
    <button onClick={increaseCountVar}>Var 증가</button>
  </div>
);
```

Hook – useRef()

- **DOM 요소 직접 접근**

  페이지가 로딩되면 포커스가 자동으로 위치하도록 함

이름을 입력하세요 전송

```
const ref = useRef(initialValue);
```

```
<input ref={ref} />
```

Hook – useRef()

- **DOM 요소 직접 접근 – InputFocus.jsx**

```
function InputFocus() {
  // useRef 훅을 사용하여 input 요소에 대한 참조를 생성
  const inputRef = useRef(null);
  console.log(inputRef); // {current: null} 객체 - 초기값은 null입니다.

  useEffect(() => {
    console.log(inputRef);
    // 컴포넌트가 마운트된 후 input 요소에 포커스를 설정
    inputRef.current.focus();
  }, []);
```

```
▼ {current: input}
  ▼ current: input
      value: (...)
    ▶ __reactEvents$vm035nkh09r: Set(1) {
    ▶ __reactFiber$vm035nkh09r: FiberNode
```

Hook – useRef()

- **DOM 요소 직접 접근 – InputFocus.jsx**

```
const handleSubmit = () => {
  // input 요소의 현재 값을 알림으로 표시
  alert(`환영합니다. ${inputRef.current.value}님`);

  // 버튼 클릭 후에도 input 요소에 포커스 유지
  inputRef.current.focus();
  // 버튼 클릭 후 input 요소의 값을 초기화
  inputRef.current.value = "";
};

return (
  <div>
    {/* ref 속성은 특정 DOM 요소에 접근할 수 있게 해줍니다. */}
    <input
      type="text"
      ref={inputRef}
      placeholder="이름을 입력하세요"
    />
    <button onClick={handleSubmit}>전송</button>
  </div>
);
```

Ref 속성이 있어야
{current: input}

Hook – useCallback()

- **메모이제이션(Memoization) 이란?**

  한 번 계산(또는 생성)한 결과를 저장해두고, 다음에 같은 조건이면 다시 계산하지 않고 재사용하는 것

```
1 + 1 = 2
```

처음엔 계산

☞ 다음엔 그냥 "2"를 기억해서 바로 사용

```
const handleClick = () => {
  console.log("클릭");
};
```

렌더링될 때마다 새로운 함수 생성됨(주소가 바뀜)

Hook – useCallback()

- **useCallback()**

  함수를 "재사용(캐싱)"해서 불필요한 재생성을 막는 리액트 훅입니다.

```
const memoizedFn = useCallback(() => {
  실행코드;
}, [item]);
```

렌더링 되어도 같은 함수 재사용(캐싱) – 초기화되지 않음

구조

- 첫번째 인자: 메모이제이션 해줄 콜백함수
- 두번째 인자: 의존성 배열

Hook – useCallback()

- **useCallback()**

number: 4

4 클릭

```
handleClick 함수가 변경되었습니다.
handleClick 함수가 변경되었습니다.
4 handleClick 함수가 변경되었습니다.
number= 4
handleClick 함수가 변경되었습니다.
handleClick 함수가 변경되었습니다.
```

Hook – useCallback()

- **useCallback()**

```
function UseCallbackExample() {
  const [number, setNumber] = useState(0);
  /* 클릭하지 않아도 handleClick이 계속 변경되는 이유는
  컴포넌트가 리렌더링될 때마다 handleClick 함수가 새로 생성되기 때문임.
  useCallback 훅을 사용하여 handleClick 함수를 메모이제이션하면,
  number가 변경될 때만 handleClick 함수가 새로 생성되고,
  그렇지 않을 때는 이전에 생성된 함수를 재사용하게 됩니다. */

  /*const handleClick = () => {
    console.log("number=", number);
  }*/

  const handleClick = useCallback(() => {
    console.log("number=", number);
  }, [number]); // number가 변경될 때만 handleClick 함수가 새로 생성됩니다.
```

Hook – useCallback()

- **useCallback()**

```
  useEffect(() => {
    console.log("handleClick 함수가 변경되었습니다.");
  }, [handleClick]); // handleClick이 변경될 때마다 이 useEffect가 실행

  return (
    <div>
      <h3>number: {number}</h3>
      <input
        type="number"
        value={number}
        onChange={(e) => setNumber(Number(e.target.value))}
      />
      <button onClick={handleClick}>클릭</button>
    </div>
  );
}
```

Hook – useMemo()

- **useMemo란?**

리액트에서 연산 비용이 큰 값을 "메모이제이션(캐싱)"해서 성능을 최적화 하는 훅입니다.

컴포넌트는 **렌더링될 때마다 함수가 다시 실행**됩니다.

→ 이때 무거운 계산도 매번 다시 실행됨

☞ **useMemo**를 사용하면

의존성이 바뀌지 않는 한 이전 계산 결과를 재사용합니다.

```
const memoizedValue = useMemo(() => {
  return 계산값;
}, [의존성]);
```

- 첫 번째 인자: 계산 함수
- 두 번째 인자: 의존성 배열
- 의존성이 바뀔 때만 다시 계산

상태 관리 – useReducer()

- **useReducer() 함수**

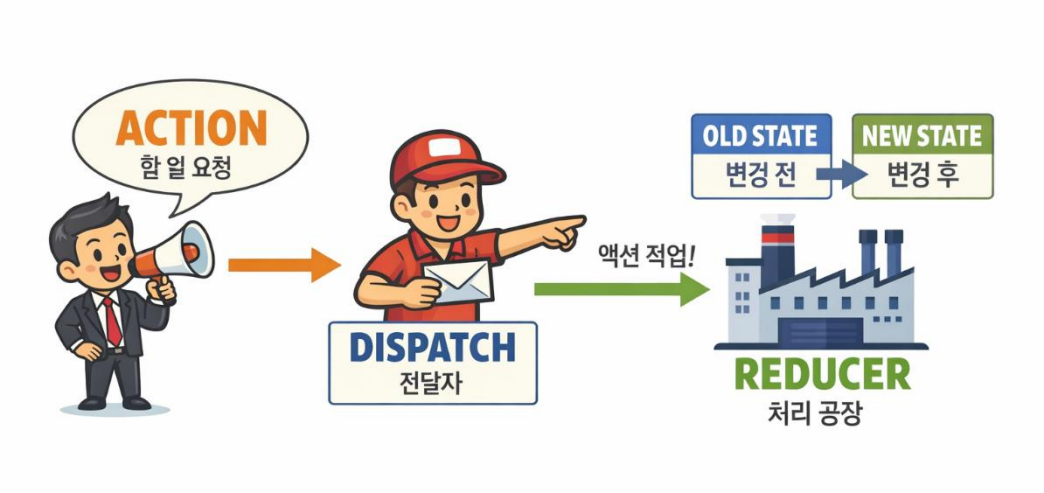
리액트의 useReducer는 상태(state)를 더 **체계적으로 관리하기 위한 훅**입니다. 특히 상태 변화 로직이 복잡할 때 useState보다 훨씬 유용합니다.
즉, **상태 변경 로직을 함수로 분리해서 관리**하는 상태 관리 훅입니다.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

구성 요소

- **state** → 현재 상태
- **dispatch** → 상태를 변경하기 위해 action을 전달하는 함수
- **reducer** → 현재 상태와 액션을 받아 새로운 상태를 반환하는 함수
- **initialState** → 초기 상태

상태 관리 – useReducer()



- **reducer() 함수 구조**

```
const reducer = (state, action) => {

};
```

  - 반드시 이전 state를 기반으로 새로운 state 반환

상태 관리 – useReducer()

- **useState vs useReducer**

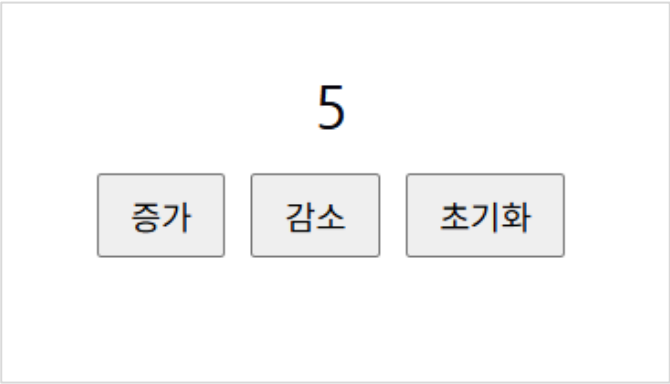
구분	useState	useReducer
사용 난이도	쉬움	조금 어려움
상태 구조	단순	복잡
로직 관리	컴포넌트 내부	reducer로 분리
유지보수	보통	매우 좋음

useReducer가 좋은 상황

- 상태가 **객체/배열로 복잡할 때**
- 상태 변경 로직이 **여러 곳에서 반복될 때**
- **조건 분기가 많을 때 (if, switch)**
- 폼, 장바구니, 인증 상태 관리 등

상태 관리 – useReducer()

- **useReducer 사용 예제**




```
▼ {count: 0}                    ▶ {type: 'INCREMENT'}
    count: 0
  ▶ [[Prototype]]: Object
▼ {count: 1}                    ▼ {type: 'INCREMENT'}
    count: 1                        type: "INCREMENT"
  ▶ [[Prototype]]: Object         ▶ [[Prototype]]: Object
▶ {count: 2}   ▶ {type: 'INCREMENT'}
▶ {count: 3}   ▶ {type: 'INCREMENT'}
```

상태 관리 – useReducer()

- **useReducer 사용 예제 – CountReducer.jsx**

```
import React, { useReducer } from "react";

/* reducer 함수는 현재 상태와 액션을 받아
   새로운 상태를 반환하는 함수입니다. */
const reducer = (state, action) => {
  console.log(state, action);

  // 액션의 타입에 따라 상태를 업데이트하는 로직을 구현합니다.
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    case "RESET":
      return {count: 0};
    default:
      return state;
  }
};
```

상태 관리 – useReducer()

- **useReducer 사용 예제 – CountReducer.jsx**

```
const CounterReducer = () => {
  // useReducer 훅을 사용하여 상태와 디스패치 함수를 생성합니다.
  // initialState는 { count: 0 }으로 설정되어 있습니다.
  const [state, dispatch] = useReducer(reducer, {count: 0});

  return (
    <div>
      <h2>{state.count}</h2>
      <button onClick={() =>
        dispatch({ type: "INCREMENT" })}>증가</button>
      <button onClick={() =>
        dispatch({ type: "DECREMENT" })}>감소</button>
      <button onClick={() =>
        dispatch({ type: "RESET" })}>초기화</button>
    </div>
  );
};
```

상태 관리 – useReducer()

- 은행 입, 출금 거래

현재 잔액: 2000원

3000 예금 출금

상태 관리 – useReducer()

- **은행 입, 출금 거래 – BankReducer.jsx**

```
import React, { useReducer, useState } from 'react';

const bankReducer = (state, action) => {
  console.log("reducer가 작동합니다.", state, action);

  switch (action.type) {
    // 예금 시에는 현재 잔액에 액션의 payload를 더하여 새로운 상태를 반환합니다.
    case 'DEPOSIT':
      return { ...state, balance: state.balance + action.payload };
    // 출금 시에는 현재 잔액에서 액션의 payload를 빼서 새로운 상태를 반환합니다.
    case 'WITHDRAW':
      if (state.balance < action.payload) {
        alert("잔액이 부족합니다.");
        return state; // 잔액이 부족할 경우 상태를 변경하지 않고 그대로 반환합니다.
      }
      return { ...state, balance: state.balance - action.payload };
    default:
      return state; // 현재 상태를 그대로 반환합니다.
  }
};
```

상태 관리 – useReducer()

- **은행 입, 출금 거래 – BankReducer.jsx**

```
const BankReducer = () => {
    const [amount, setAmount] = useState(0);
    // 초기 잔액을 0으로 설정합니다.
    const [state, dispatch] = useReducer(bankReducer, { balance: 0 });

  return (
    <div>
      <h3>현재 잔액: {state.balance}원</h3>
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(parseInt(e.target.value))}
        step="1000"
      />
      <button onClick={() => {dispatch({ type: 'DEPOSIT', payload: amount })}}>예금</button>
      <button onClick={() => {dispatch({ type: 'WITHDRAW', payload: amount })}}>출금</button>
    </div>
  )
}
```

전역 공유 – useContext()

- **useContext()**

  리액트의 `useContext`는 **컴포넌트 트리 전체에서 데이터를 쉽게 공유하기 위한 훅**입니다.

  쉽게 말하면 **props 없이 데이터 전달하는 방법**입니다.

  ✓ props drilling 문제 - 계속 전달해야 함

```
<Parent user={user} />
  <Child user={user} />
    <GrandChild user={user} />
```

전역 공유 – useContext()

- **useContext() 3단계**

1. Context 생성

```
import { createContext } from "react";

export const UserContext = createContext();
```

2. Provider로 감싸기

```
<UserContext.Provider value={user}>
  <App />
</UserContext.Provider>
```

3. 값 사용

```
const Profile = () => {
  const user = useContext(UserContext);

  return <h2>{user.name}</h2>;
};
```

전역 공유 – useContext()

- **Props – user 객체 전달**

User 컴포넌트입니다.

사용자 정보를 Context로 전달합니다.

UserChild 컴포넌트입니다.

사용자 이름: 이순신
사용자 나이: 45

UserGrandChild 컴포넌트입니다.

사용자 이름: 이순신
사용자 나이: 45

전역 공유 – useContext()

- **Props – ParentProps.jsx**

```
import ChildProps from "./ChildProps";

const ParentProps = () => {
  const user = {
    name: "이순신",
    age: 45,
  };

  return (
    <div className="user">
      <h2>ParentProps 컴포넌트입니다.</h2>
      <ChildProps user={user} />
    </div>
  );
}
```

전역 공유 – useContext()

- **Props – ChildProps.jsx**

```
import GrandChildProps from "./GrandChildProps";

const ChildProps = ({ user }) => {
  return (
    <div className="user-child">
      <h2>ChildProps 컴포넌트입니다.</h2>
      <p>사용자 이름: {user.name}</p>
      <p>사용자 나이: {user.age}</p>
      <GrandChildProps user={user} />
    </div>
  );
}

export default ChildProps;
```

전역 공유 – useContext()

- **Props – GrandChildProps.jsx**

```
const GrandChildProps = ({ user }) => {
  return (
    <div className="user-grandchild">
      <h3>GrandChildProps 컴포넌트입니다.</h3>
      <p>사용자 이름: {user.name}</p>
      <p>사용자 나이: {user.age}</p>
    </div>
  );
}

export default GrandChildProps;
```

전역 공유 – useContext()

- **useContext() 사용 예제 – user 객체 전달**

User 컴포넌트입니다.

사용자 정보를 Context로 전달합니다.

UserChild 컴포넌트입니다.

사용자 이름: 이순신
사용자 나이: 45

UserGrandChild 컴포넌트입니다.

사용자 이름: 이순신
사용자 나이: 45

전역 공유 – useContext()

- **useContext() – User.jsx**

```
// UserContext를 생성합니다. 기본값은 빈 객체입니다.
export const UserContext = createContext({});

const User = () => {
  // user 데이터를 정의합니다.
  const user = {
    name: "이순신",
    age: 45,
  };

  return (
    <div className='user'>
      <h2>User 컴포넌트입니다.</h2>
      <p>사용자 정보를 Context로 전달합니다.</p>

      {/* UserContext.Provider로 감싸서 user 값을
                  하위 컴포넌트에 제공합니다 */}
      <UserContext.Provider value={user}>
        <UserChild />
      </UserContext.Provider>
    </div>
  );
```

전역 공유 – useContext()

- **useContext() – UserChild.jsx**

```
import { useContext } from 'react';
import { UserContext } from './User';
import UserGrandChild from './UserGrandChild';

const UserChild = () => {
  // UserContext에서 user 데이터를 가져옵니다.
  const user = useContext(UserContext);
  console.log(user);

  return (
    <div className='user-child'>
      <h3>UserChild 컴포넌트입니다.</h3>
      <p>사용자 이름: {user.name}</p>
      <p>사용자 나이: {user.age}</p>

      <UserGrandChild />
    </div>
  );
}
```

전역 공유 – useContext()

- **useContext() – UserGrandChild.jsx**

```
import { useContext } from 'react';
import { UserContext } from './User';

const UserGrandChild = () => {
  // UserContext에서 user 데이터를 가져옵니다.
  const user = useContext(UserContext);

  return (
    <div className='user-grandchild'>
      <h4>UserGrandChild 컴포넌트입니다.</h4>
      <p>사용자 이름: {user.name}</p>
      <p>사용자 나이: {user.age}</p>
    </div>
  );
}

export default UserGrandChild;
```

전역 공유 – useContext()

- useContext() 사용 예제 – 테마 변경

전역 공유 – useContext()

- **useContext() 사용 예제 – ParentTheme.jsx**

```
import { createContext, useState } from "react"
import ChildTheme from "./ChildTheme";

// Context 생성
export const ThemeContext = createContext("light");

const ParentTheme = () => {
  const [currentTheme, setCurrentTheme] = useState("light");

  // 테마 토글 함수
  const toggleTheme = () => {
    setCurrentTheme(prevTheme => (
      prevTheme === 'light' ? 'dark' : 'light'
    ))
  }
```

전역 공유 – useContext()

- **useContext() 사용 예제 – ParentTheme.jsx**

```
  // 부모 컴포넌트 스타일
  const parentStyle = {
    background: currentTheme === 'dark' ? '■#333' : '□#fff',
    color: currentTheme === 'dark' ? '□#fff' : '■#333',
    padding: '20px'
  }

  // 부모 컴포넌트 렌더링
  return(
    <div style={parentStyle}>
      <h3>현재 테마: {currentTheme}</h3>
      {/* 하위 컴포넌트에 Context 제공 */}
      <ThemeContext.Provider value={{currentTheme, toggleTheme}}>
        <ChildTheme />
      </ThemeContext.Provider>
    </div>
  )
}
```

전역 공유 – useContext()

- **useContext() 사용 예제 – ChildTheme.jsx**

```
import { useContext } from "react"
import { ThemeContext } from "./ParentTheme"

const ChildTheme = () => {
  // Context 사용
  const { currentTheme, toggleTheme } = useContext(ThemeContext);
  console.log("현재 테마:", currentTheme);

  // 자식 컴포넌트 스타일
  const childStyle = {
    background: currentTheme === 'dark' ? '⬛#555' : '⬜#f0f0f0',
    color: currentTheme === 'dark' ? '⬜#fff' : '⬛#333',
    padding: '15px',
  };
```

전역 공유 – useContext()

- **useContext() 사용 예제 – ChildTheme.jsx**

```
  // 자식 컴포넌트 렌더링
  return(
    <div style={childStyle}>
      <h4>Child 컴포넌트</h4>
      <p>현재 테마: <strong>{currentTheme}</strong></p>
      <button onClick={toggleTheme}>
        테마 변경
      </button>
    </div>
  )
}

export default ChildTheme
```